SIMULATION RUNTIME · V0.1.0

仿真平台使用手册

SO-101 / MuJoCo 独立仿真服务 · HTTP API · Python SDK · 浏览器查看

协议版本：simulation-runtime/v1

适用对象：EmbodiedEvo 及其他接入项目、评测与调试人员

代码仓库：https://github.com/lihuigang/MuJoCo

文档日期：2026-10-19

单机部署 · 项目级隔离 · 双相机观测 · 异步可靠操作

目录

1 平台概览与能力边界

2 架构与会话模型

3 快速开始（本机 / Docker）

4 浏览器查看仿真画面

5 Python SDK 使用

6 HTTP API 摘要

7 会话语义、预算与可靠性

8 配置项

9 部署与安全

10 故障排查

11 测试、版本与支持

一分钟上手：在 `deploy/` 下执行 `cp tokens.example.json tokens.json`（改成自己的令牌）并运行 `docker compose up -d --build`，然后打开 `http://127.0.0.1:8080/viewer`，输入项目令牌即可看到仿真画面。

1. 平台概览与能力边界

simulation-runtime 是一个独立、项目化的机器人仿真服务。它把 SO-101 / MuJoCo 的 `pick_cube` 任务从 EmbodiedEvo 中解耦出来，任何项目只需 HTTP 调用或 Python SDK 即可使用，无需复制 EmbodiedEvo 仓库、安装 MuJoCo 资产或共享宿主机目录。

已具备的能力

- **两种动作模式**：`typed` 逐步提交关节或 IK 动作；`exec` 提交受限 Python 程序，会话内全局变量持久化。
- **两个阶段**：`playground` 允许重置并续满动作预算；`test` 禁止重置，用于正式评测。
- **任务规格**：300 次原生动作上限、20 Hz、每次动作 10 个物理步、双 256×256 相机（`top_cam`、`wrist_cam`）、5 帧稳定成功判定。
- **多项目隔离**：Bearer 项目令牌映射到项目身份，会话、操作、观测与工件全部按项目隔离。
- **可靠操作**：长操作异步化（`202` + operation 轮询）、幂等键、断线重试安全；超时或崩溃后标记 `outcome_uncertain`，动作绝不重放。
- **浏览器查看**：内置 `/viewer` 页面，轮询显示双相机画面与预算状态。

第一阶段边界

- 单机部署（Docker Compose 或 systemd），SQLite + 本地工件目录，不支持多副本水平扩展。
- 每个会话独占一个 worker 进程，但尚未实现每会话容器 / cgroup 资源限额。
- `exec` 是协作式沙箱，**不是不可信代码的安全边界**，仅向可信团队发放 `exec` 权限。
- 服务自身不终止 TLS，对外必须由 nginx / 网关反向代理；不支持 Kubernetes 调度。
- viewer 为 PNG 轮询快照，没有视频流、视频工件与失败自动回放。

2. 架构与会话模型



关键语义

概念	说明
`phase=playground`	允许 `/reset`；每次重置恢复同一 seed 场景，并把本次尝试的原生动作预算续满。累计步数仍保留。
`phase=test`	禁止 `/reset`（返回 403）；用于一次性的正式评测。
`mode=typed`	动作结构为 5 个关节目标 + 1 个夹爪，或末端位置 + 夹爪；只能调用 `/steps`。
`mode=exec`	只能调用 `/executions`；程序内可使用 `get_instruction`、`get_observation`、`step`、`reset` 及受限 `numpy` / `math`。
预算	会话创建时确定原生动作上限（300）与 exec 执行次数预算；耗尽后返回 `budget_exhausted`。
操作	每次写请求生成 operation。同步等待超时返回 `202`，客户端用 `Location` 轮询；同一幂等键重试返回同一操作。
失败	worker 超时/崩溃会中断会话并标记 `outcome_uncertain=true`，服务不会重放动作，客户端需新建会话。

成功判定

红色方块需被放到绿色目标垫上、松开夹爪，并连续 5 帧保持静止（方块中心距目标中心 ⩽ 4.5 cm、高度 < 5 cm、线速度 < 0.08 m/s）。

3. 快速开始

3.1 本机 Python 运行

要求 Python 3.10+；macOS 直接渲染，Linux 需要 EGL 或 OSMesa。

```
git clone git@github.com:lihuigang/MuJoCo.git
cd MuJoCo
python -m venv .venv
.venv/bin/pip install -e '.[service,mujoco]'

export SIM_RUNTIME_TOKENS_JSON='[{
  "token": "replace-with-a-random-token-at-least-16-chars",
  "project_id": "demo",
  "scopes": ["playground", "test", "typed", "exec"]
}]'
export SIM_RUNTIME_DATA_DIR="./data"
.venv/bin/simulation-runtime serve
```

服务默认监听 `127.0.0.1:8080`。验证：

```
export SIM_RUNTIME_TOKEN='replace-with-a-random-token-at-least-16-chars'
.venv/bin/simulation-runtime capabilities
.venv/bin/simulation-runtime backend-smoke --backend mujoco_so101
.venv/bin/simulation-runtime smoke --output frame.png
```

3.2 Docker Compose 运行

```
cd deploy
cp tokens.example.json tokens.json   # 写入真实随机令牌
docker compose up -d --build
docker compose logs -f simulation-runtime
```

镜像名 `simulation-runtime:0.1.0`，容器内以非 root 用户运行，数据保存在命名卷。compose 默认把端口绑定到 `127.0.0.1:8080`，健康检查使用 `/readyz`。

> **远程服务器访问：**推荐 SSH 隧道 `ssh -L 8080:127.0.0.1:8080 user@server`，然后在本地浏览器打开页面；或通过 nginx 反向代理提供 HTTPS。不要把明文端口直接暴露到公网。

3.3 验证服务

```
curl -s http://127.0.0.1:8080/healthz
curl -s -H "Authorization: Bearer $SIM_RUNTIME_TOKEN" \
  http://127.0.0.1:8080/v1/capabilities
```

交互式 API 文档位于 `http://127.0.0.1:8080/docs`（Swagger UI）。

4. 浏览器查看仿真画面

服务内置 `/viewer` 页面（访问 `/` 会自动跳转），无需安装前端依赖。输入项目令牌后，可选择已有会话或新建 playground 会话；页面每 0.5–2 秒拉取最新观测，并排显示两路相机，同时展示成功状态、动作预算与观测序号，并支持重置和关闭会话。

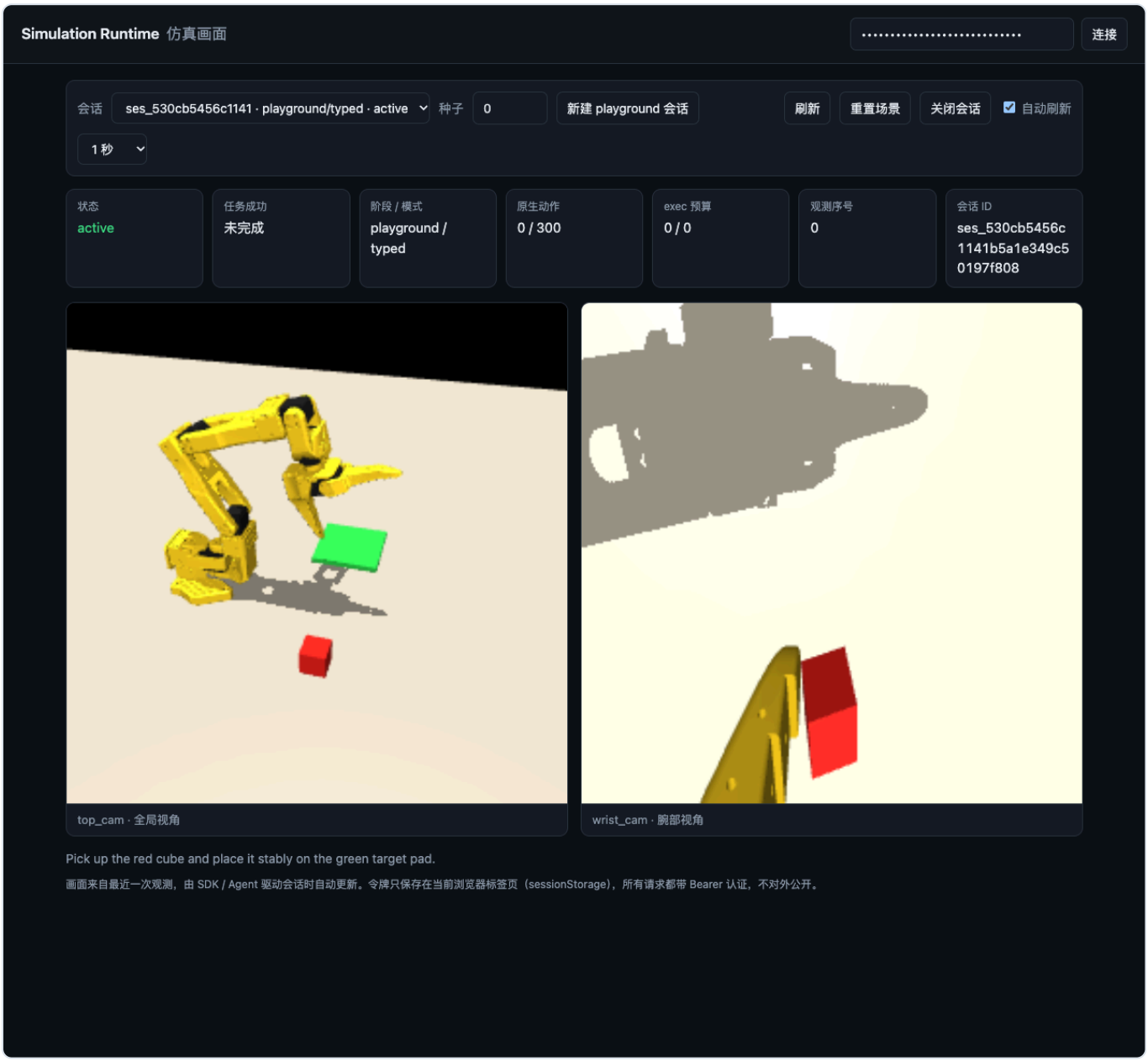


图 1：/viewer 页面，双相机画面与状态卡片

- 页面本身只返回静态 HTML，所有数据请求仍带 `Authorization: Bearer`；令牌只保存在当前标签页的 `sessionStorage`。
- 查看是被动的：画面随 SDK / Agent 驱动的会话更新，页面不会自动步进。
- 页面设置了 CSP、`X-Frame-Options: DENY`、`Referrer-Policy: no-referrer`，不加载任何第三方资源。

5. Python SDK 使用

SDK 仅使用标准库，安装本包即可使用。

5.1 typed：逐步提交动作

```
from simulation_runtime.sdk import Client

client = Client("http://127.0.0.1:8080", token="<项目令牌>")

with client.session(
    backend="mujoco_so101", task="pick_cube",
    phase="playground", mode="typed", seed=0,
) as session:
    print(session.instruction())
    observation = session.observation()
    print(observation["state"]["ee_pose"])

    # 下载相机帧
    session.download_camera(observation, "top_cam", path="top.png")

    # IK 动作：末端位置 + 夹爪开合（0=闭合，1=张开）
    result = session.step({
        "ee_position": [0.2, -0.1, 0.10],
        "ee_joint_state": [1.0],
    })
    print(result["success"], result["native_steps_remaining"])
```

5.2 exec：受限 Python 程序

```
with client.session(mode="exec", exec_budget=100, seed=0) as session:
    result = session.execute("""
obs = get_observation()
for repeat in range(20):
    step({'ee_position': [0.2, -0.1, 0.08], 'ee_joint_state': [1.0]})
print(get_instruction())
""")
    print(result["ok"], result["stdout"],
          result["native_steps_used"])
```

安全提示：exec 程序在仿真 worker 内执行，AST 校验与受限内置函数只用于防误操作，不构成对不可信代码的隔离。仅向可信团队发放 `exec` 权限。

5.3 观测与工件

每次观测包含 `instruction`、`state`（关节角、夹爪、末端位姿）、`action`（最近一次指令）、`control_hz` 与 `cameras`。相机帧以 PNG 保存，通过 `href` / `artifact_id` 下载；跨项目访问返回 404。

6. HTTP API 摘要

除静态页面与探针外，所有 `/v1` 接口要求 `Authorization: Bearer <project-token>`。

方法	路径	说明
GET	`/` → `/viewer`	浏览器查看页（页面无鉴权，数据请求需令牌）
GET	`/healthz` / `/readyz`	存活 / 就绪探针
GET	`/v1/capabilities`	后端、任务、动作 schema、限制
POST	`/v1/sessions`	创建会话，返回会话与初始观测
GET	`/v1/sessions`	列出本项目会话
GET	`/v1/sessions/{id}`	会话状态与预算
GET	`/v1/sessions/{id}/observation`	最新观测元数据
POST	`/v1/sessions/{id}/steps`	typed 动作（需要 typed scope）
POST	`/v1/sessions/{id}/executions`	exec 程序（需要 exec scope）
POST	`/v1/sessions/{id}/reset`	仅 playground 且需要对应 phase scope
POST	`/v1/sessions/{id}/close`	关闭会话
GET	`/v1/sessions/{id}/operations[/{op}]`	操作历史 / 单个操作轮询
GET	`/v1/artifacts/{id}`	下载 PNG（项目隔离）

写操作默认同步等待最多 `request_wait_sec`；超时返回 `202 Accepted`，带 `Location` 与 `Retry-After`。支持 `Idempotency-Key` 请求头（≤200 字符）。

> **错误格式：**统一为 `{"error": {"code": "...", "message": "..."}}`。操作内部失败通常以 `200` + `status=failed` 返回，便于审计。

7. 会话语义、预算与可靠性

场景	行为
playground 重置	恢复同一 seed 场景，动作预算续满，累计 `native_steps_used` 保留，观测序号继续递增。
test 重置	拒绝：`403 forbidden`。
模式混用	typed 会话调用 `/executions`（或反之）返回 `409 action_mode_mismatch`。
预算耗尽	返回 `409 budget_exhausted`；playground 可重置后续期。
同会话并发	同一会话的写操作串行化；进行中再次提交返回 `409 session_busy`。
幂等重试	相同幂等键 + 相同请求返回原操作；相同键 + 不同请求返回 `409 idempotency_conflict`。
exec 超时	中断会话，操作标记失败且 `outcome_uncertain=true`；不重放，客户端应新建会话。
会话回收	空闲超过 `session_idle_ttl_sec` 或超过最大生命周期会被回收；服务重启会把活动会话标记为中断。

推荐评测流程：每个 episode 新建会话 → 执行策略 → 读取最终观测与预算 → 关闭会话。测试阶段使用 `test` 防止重置，playground 仅用于调试。

8. 配置项

环境变量	默认值	说明
`SIM_RUNTIME_TOKENS_FILE` / `TOKENS_JSON`	必填	项目令牌列表（文件或内联 JSON）
`SIM_RUNTIME_DATA_DIR`	`./data`	SQLite 与工件目录
`SIM_RUNTIME_BACKENDS`	`mujoco_so101`	启用的后端，逗号分隔（测试可用 `fake`）
`SIM_RUNTIME_HOST` / `PORT`	`127.0.0.1` / `8080`	监听地址与端口
`SIM_RUNTIME_MAX_SESSIONS`	`4`	全局并发会话上限
`SIM_RUNTIME_MAX_SESSIONS_PER_PROJECT`	`2`	单项目并发会话上限
`SIM_RUNTIME_DEFAULT_EXEC_BUDGET`	`100`	exec 会话默认执行次数
`SIM_RUNTIME_MAX_EXEC_BUDGET`	`1000`	单会话执行次数上限
`SIM_RUNTIME_EXEC_TIMEOUT_SEC`	`60`	单次程序执行上限
`SIM_RUNTIME_REQUEST_WAIT_SEC`	`10`	同步等待上限，超时返回 202
`SIM_RUNTIME_SESSION_IDLE_TTL_SEC`	`900`	空闲会话回收时间
`SIM_RUNTIME_SESSION_MAX_LIFETIME_SEC`	`3600`	会话最长生命周期
`SIM_RUNTIME_FORWARDED_ALLOW_IPS`	—	信任的反向代理地址（uvicorn）
`MUJOCO_GL`	`egl`（镜像）	Linux 无显示环境渲染；EGL 不可用时改为 `osmesa`

令牌文件格式见 `deploy/tokens.example.json`；修改令牌后重启服务生效。令牌至少 16 个字符，同一项目可配置多个令牌但项目 ID 必须唯一。

9. 部署与安全

9.1 部署形态

- **Docker Compose（推荐）**：单机、数据命名卷、只读挂载令牌文件；默认仅绑定本机回环地址。
- **systemd**：非容器部署，示例单元见 `deploy/simulation-runtime.service`。
- **反向代理**：nginx 示例见 `deploy/nginx.conf.example`，负责 TLS、请求体上限与超时。

9.2 安全清单

- 为每个项目生成独立随机令牌；只向可信团队发放 `exec` scope。
- 服务进程使用专用低权限用户；数据目录仅该用户可读写。
- 对外只暴露反向代理的 HTTPS 端口；`/healthz`、`/readyz` 无鉴权但只返回最小状态。
- 会话、操作、观测、工件全部按 `project_id` 隔离；跨项目 ID 返回 404。
- 动作不重放：超时/崩溃后操作标记不确定并中断会话，避免同一物理动作执行两次。
- viewer 令牌仅存于浏览器标签页；页面不加载第三方资源并设置了 CSP。

9.3 容量与备份

每个会话一个 worker 进程，主要开销来自 MuJoCo 渲染与模型加载；按 CPU 与内存设置 `max_sessions`。SQLite 与 `artifacts/` 需一起备份；当前不会自动删除旧会话与工件，请按留存策略清理。

10. 故障排查

现象	处理
401 Unauthorized	检查 `Authorization: Bearer` 头与令牌是否在配置文件中；令牌修改后需重启服务。
403 forbidden	令牌缺少对应 scope（typed / exec / phase），或尝试在 test 阶段重置。
409 session_busy	同会话已有进行中的操作；等待 operation 完成后重试，或使用幂等键安全重试。
409 capacity_exhausted	全局或项目并发会话已满；关闭空闲会话或调整上限。
execution_timeout / session_interrupted	会话已中断且不确定动作是否执行；不要复用，新建会话重新开始。
容器启动报 EGL 错误	设置 `MUJOCO_GL=osmesa`（镜像已含 libosmesa6）；或挂载可用的 GPU 驱动。
`/readyz` 返回 503	查看响应中的 `errors`：后端加载失败或 SQLite 不可写；检查数据目录权限与模型资产。
`docker compose` 命令不存在	安装 Docker Compose 插件，或使用独立的 `docker-compose` 命令。
viewer 画面不更新	页面只显示最近一次观测；需要客户端通过 SDK / API 驱动会话。确认自动刷新已开启且未被会话关闭。

11. 测试、版本与支持

11.1 本地测试

```
.venv/bin/pip install -e '.[dev]'          # 未装 MuJoCo 时相关测试自动跳过
.venv/bin/python -m pytest -q              # 当前 20 个测试
.venv/bin/pip install -e '.[dev,mujoco]'   # 真实 SO-101 渲染与 HTTP 测试
MUJOCO_GL=egl .venv/bin/python -m pytest -q
```

测试覆盖：核心会话语义与预算、受限执行、HTTP 服务（fake 后端）、MuJoCo 后端、MuJoCo over HTTP、viewer 路由。

11.2 版本

当前版本 `0.1.0`，协议 `simulation-runtime/v1`。仓库地址：https://github.com/lihuigang/MuJoCo。接口细节见仓库内 `docs/api.md`、`docs/architecture.md`、`docs/security.md` 与 `docs/deployment.md`。

11.3 许可

仓库代码许可待定。内置 SO-101 模型来自 MuJoCo Menagerie / The Robot Studio，采用 Apache-2.0；第三方声明见 `THIRD_PARTY_NOTICES.md` 与模型目录下的许可文件。